Lektion 7

Introduktion til Operativsystemer

Computerarkitektur og Operativsystemer



Emil Lykke Diget

Syddansk Universitet

eld@mmmi.sdu.dk

Lektionsoversigt

- Lektion 7: Introduktion til operativsystemer
- Lektion 8: Tråde og CPU-schedulering
- Lektion 9: Processynkronisering
- Lektion 10: Deadlocks
- Lektion 11: Memory management
- Lektion 12: Virtuel hukommelse

Outline

Hvad er et Operativsystem?

Systemkald

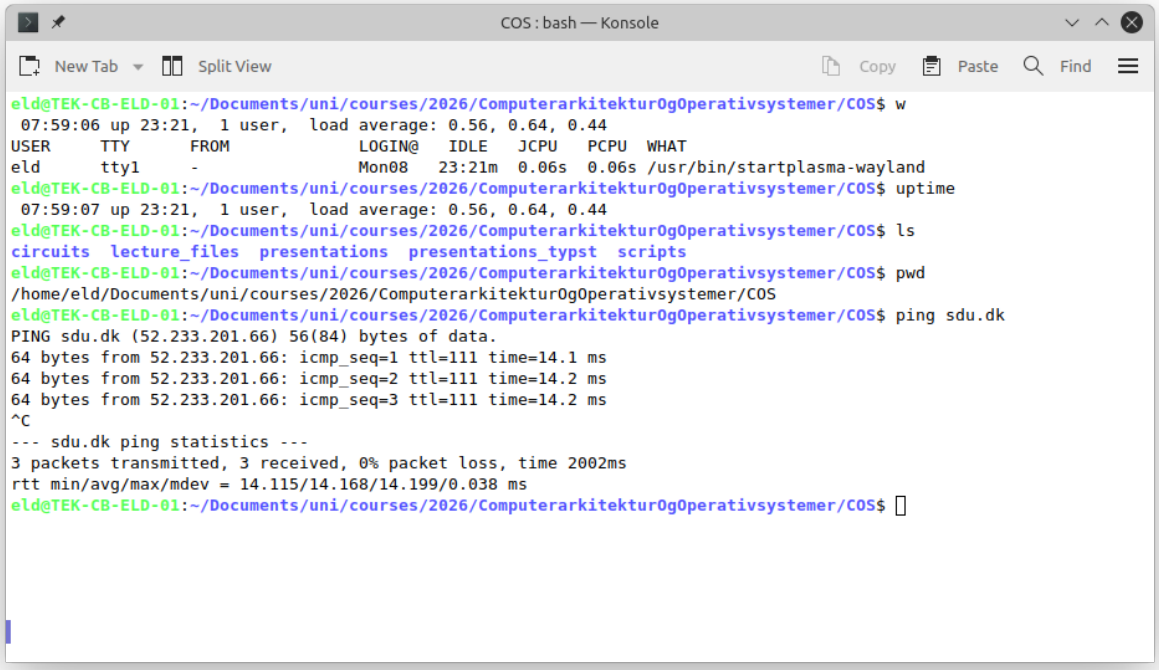
Struktur

Process

Process-schedulering

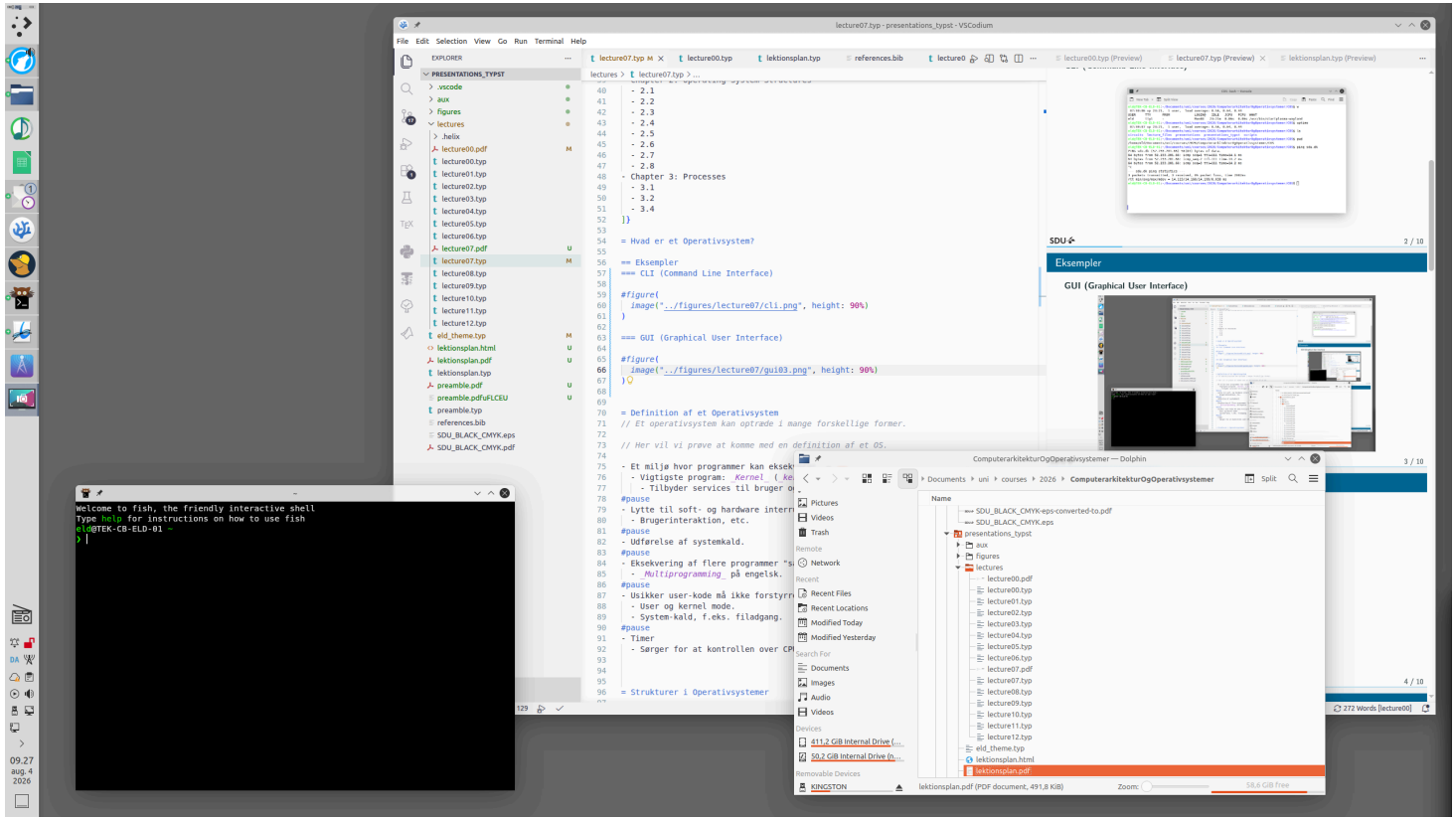
Hvad er et Operativsystem?

CLI (Command Line Interface)



Bash-shell

GUI (Graphical User Interface)



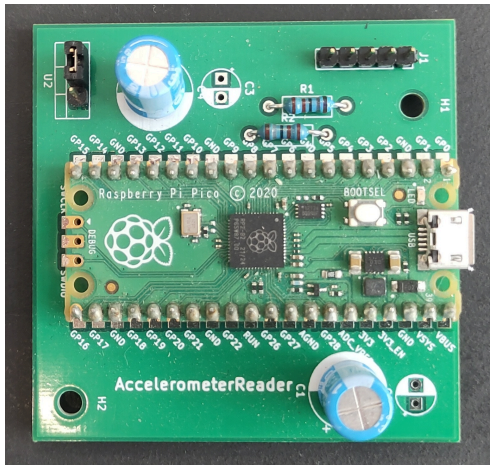
KDE Plasma 5

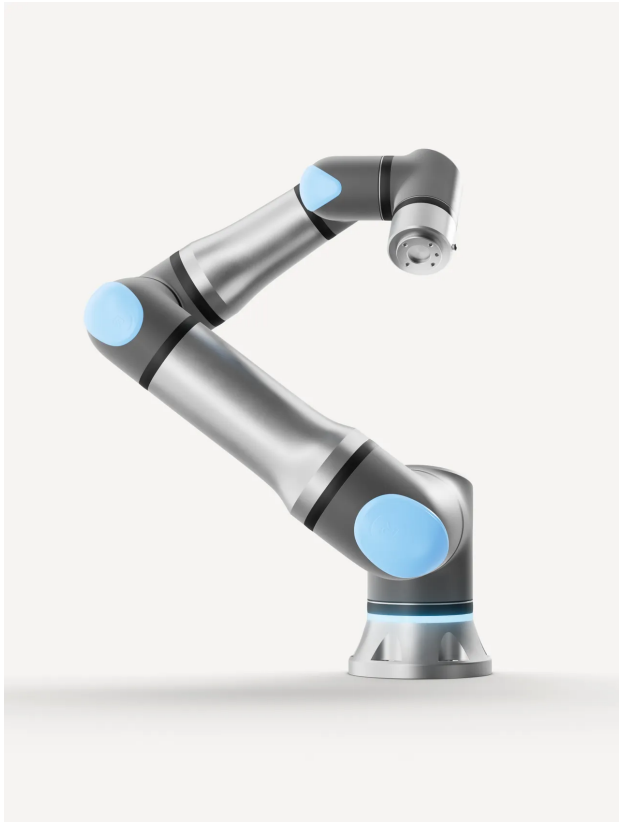
Eksempler

Uden grafisk interface

- Servere
- Embedded
- Robotter (med eller uden teach-pendant)
- osv.







Definition af et Operativsystem

Definition Et operativ-system (OS) er software, der administrerer computerens hardware.

Definition af et Operativsystem

- Et miljø hvor programmer kan eksekveres.
  - Vigtigste program: *Kernel* (*kernen* på dansk)
    - Tilbyder services til brugeren og andre programmer.

Definition af et Operativsystem

- Et miljø hvor programmer kan eksekveres.
  - Vigtigste program: *Kernel* (*kernen* på dansk)
    - Tilbyder services til brugeren og andre programmer.
- Lytte til soft- og hardware interrupts.
  - Brugerinteraktion, etc.

Definition af et Operativsystem

- Et miljø hvor programmer kan eksekveres.
  - Vigtigste program: *Kernel* (*kernen* på dansk)
    - Tilbyder services til brugeren og andre programmer.
- Lytte til soft- og hardware interrupts.
  - Brugerinteraktion, etc.
- Udførelse af *systemkald*.

Definition af et Operativsystem

- Et miljø hvor programmer kan eksekveres.
  - Vigtigste program: *Kernel* (*kernen* på dansk)
    - Tilbyder services til brugeren og andre programmer.
- Lytte til soft- og hardware interrupts.
  - Brugerinteraktion, etc.
- Udførelse af *systemkald*.
- Eksekvering af flere programmer "samtidigt"
  - *Multiprogramming* på engelsk.

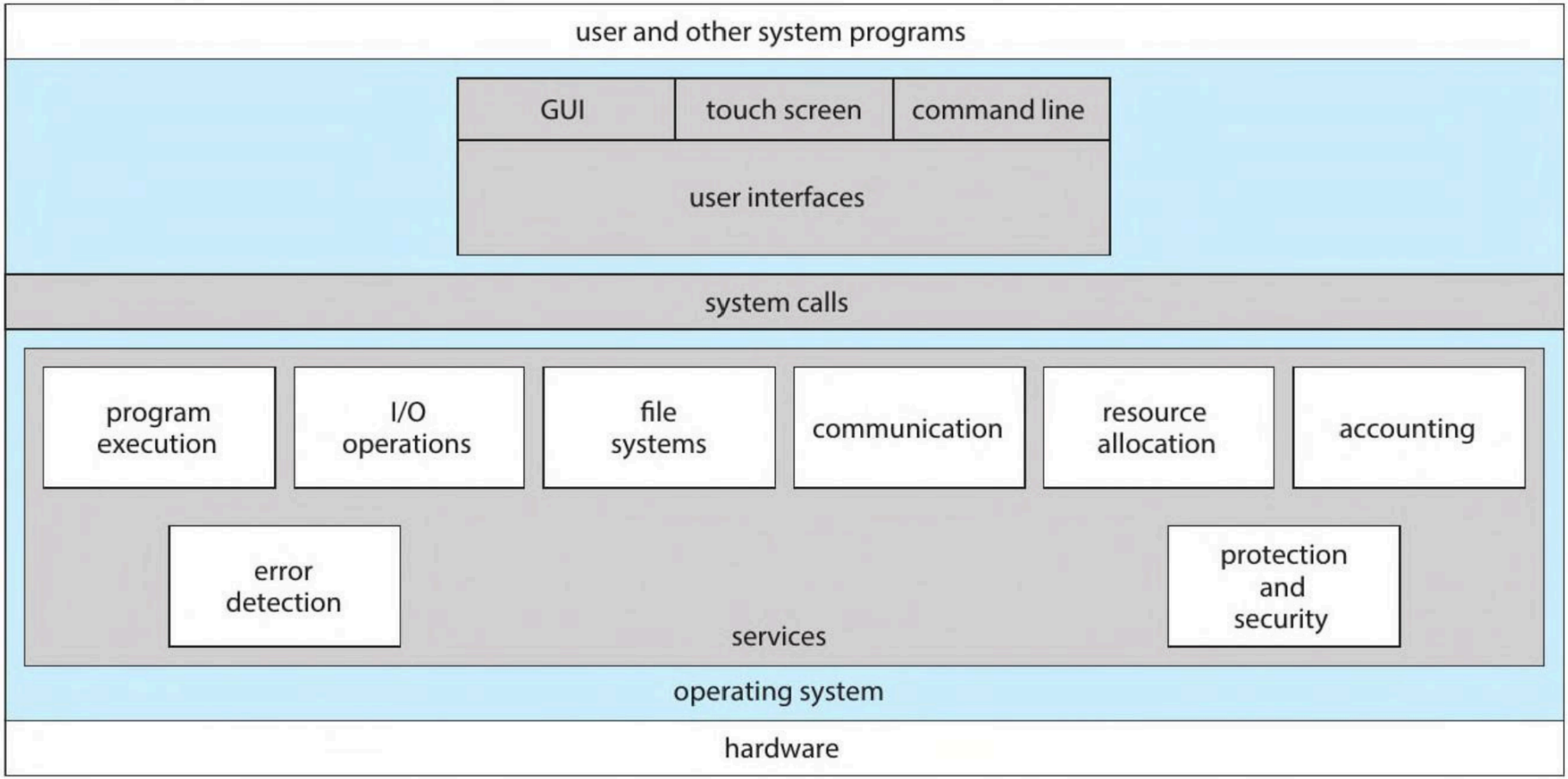
Definition af et Operativsystem

- Et miljø hvor programmer kan eksekveres.
  - Vigtigste program: *Kernel* (*kernen* på dansk)
    - Tilbyder services til brugeren og andre programmer.
- Lytte til soft- og hardware interrupts.
  - Brugerinteraktion, etc.
- Udførelse af *systemkald*.
- Eksekvering af flere programmer "samtidigt"
  - *Multiprogramming* på engelsk.
- Usikker user-kode må ikke forstyrre OSet.
  - User og kernel mode.
  - Systemkald, f.eks. filadgang.

Definition af et Operativsystem

- Et miljø hvor programmer kan eksekveres.
  - Vigtigste program: *Kernel* (*kernen* på dansk)
    - Tilbyder services til brugeren og andre programmer.
- Lytte til soft- og hardware interrupts.
  - Brugerinteraktion, etc.
- Udførelse af *systemkald*.
- Eksekvering af flere programmer "samtidigt"
  - *Multiprogramming* på engelsk.
- Usikker user-kode må ikke forstyrre OSet.
  - User og kernel mode.
  - Systemkald, f.eks. filadgang.
- Timer
  - Sørger for at kontrollen over CPU'en gives tilbage til OSet efter en given periode.

Overblik over Services



Systemkald

Systemkald

Et **systemkald** (system call) er en service, operativsystemet tilbyder computerprogrammer.

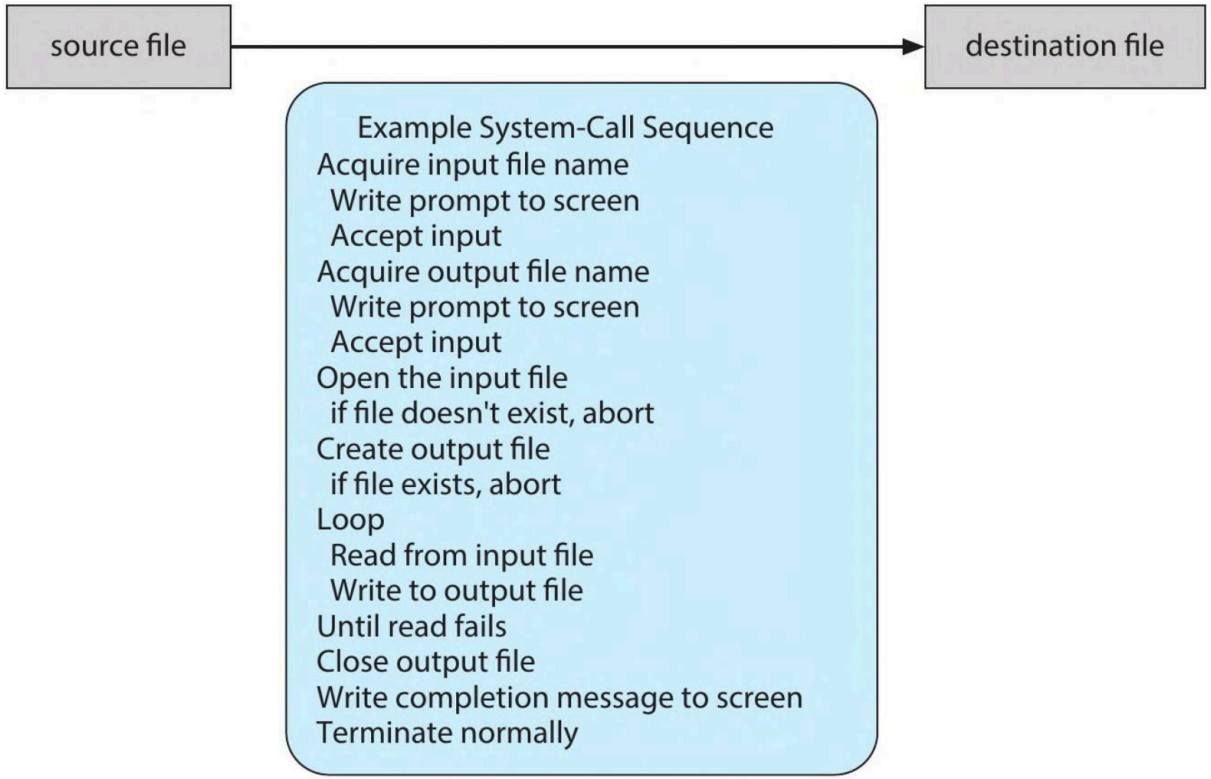
Eksempler kan f.eks. være:

- Hardwareadgang: Harddisk, webcam, etc.
- Håndtering af processer; oprettelse, terminering, etc.

I Unix og Linux er de tilgængelige igennem C-biblioteket `glibc`.

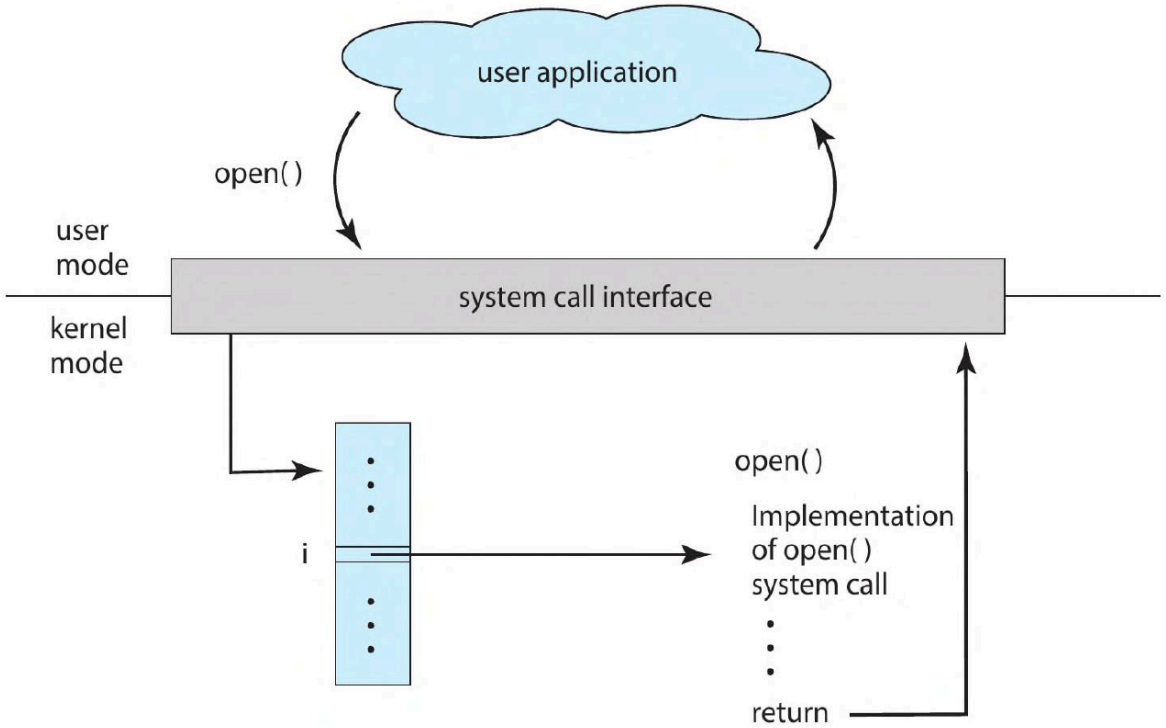
Eksempel

COPY-operation består af mange systemkald.

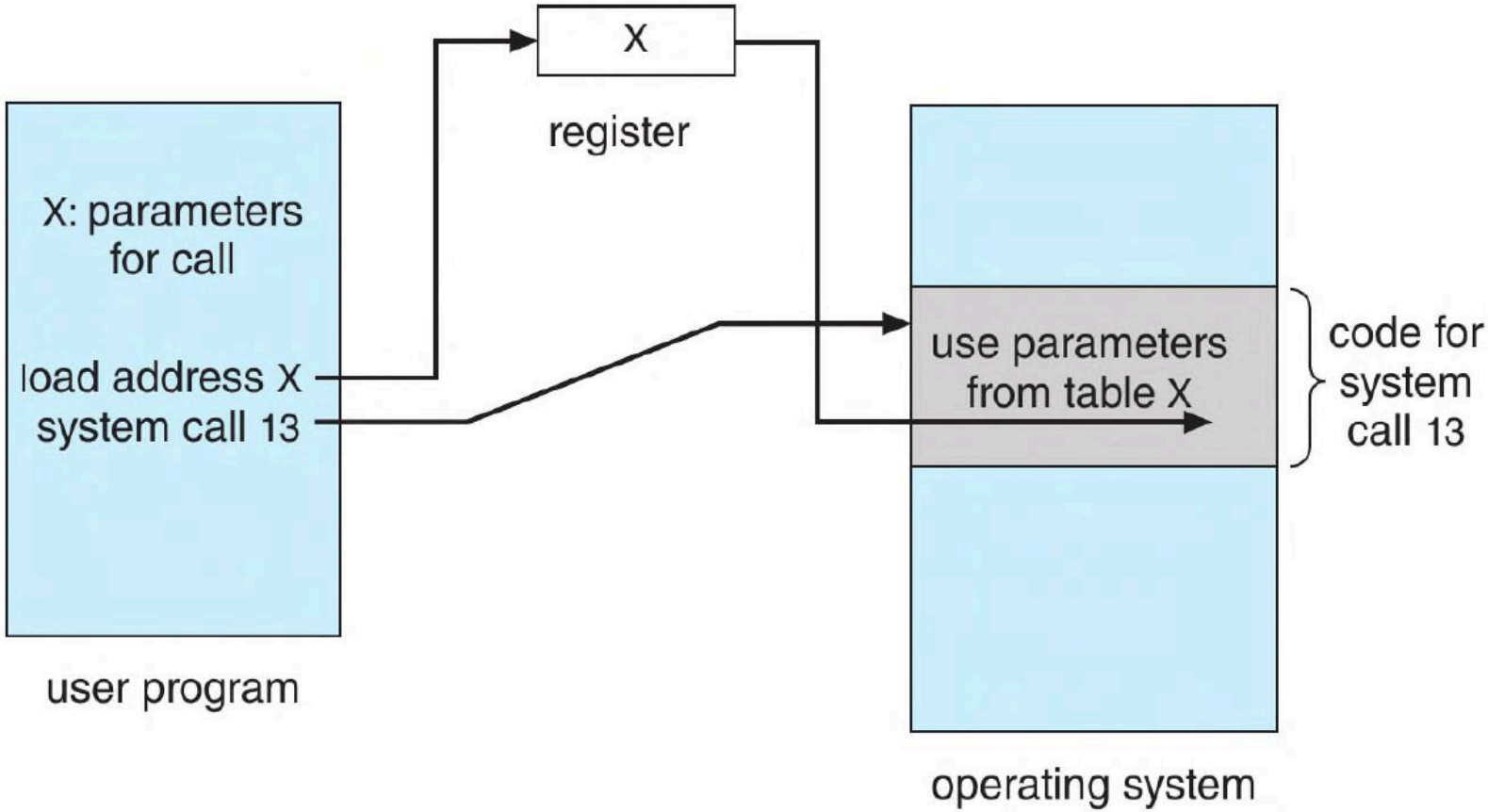


API (Application Programming Interface)

Eksempel for OPEN-operation



Parameteroverførsel til systemkald



Eksempler på systemkald

	Windows	Unix

Process control	`CreateProcess()`
	`ExitProcess()`
	`WaitForSingleObject()`
		`fork()`
		`exit()`
		`wait()`

File management	`CreateFile()`
	`ReadFile()`
	`WriteFile()`
	`CloseHandle()`
		`open()`
		`read()`
		`write()`
		`close()`

Device management	`SetConsoleMode()`
	`ReadConsole()`
	`WriteConsole()`
		`ioctl()`
		`read()`
		`write()`

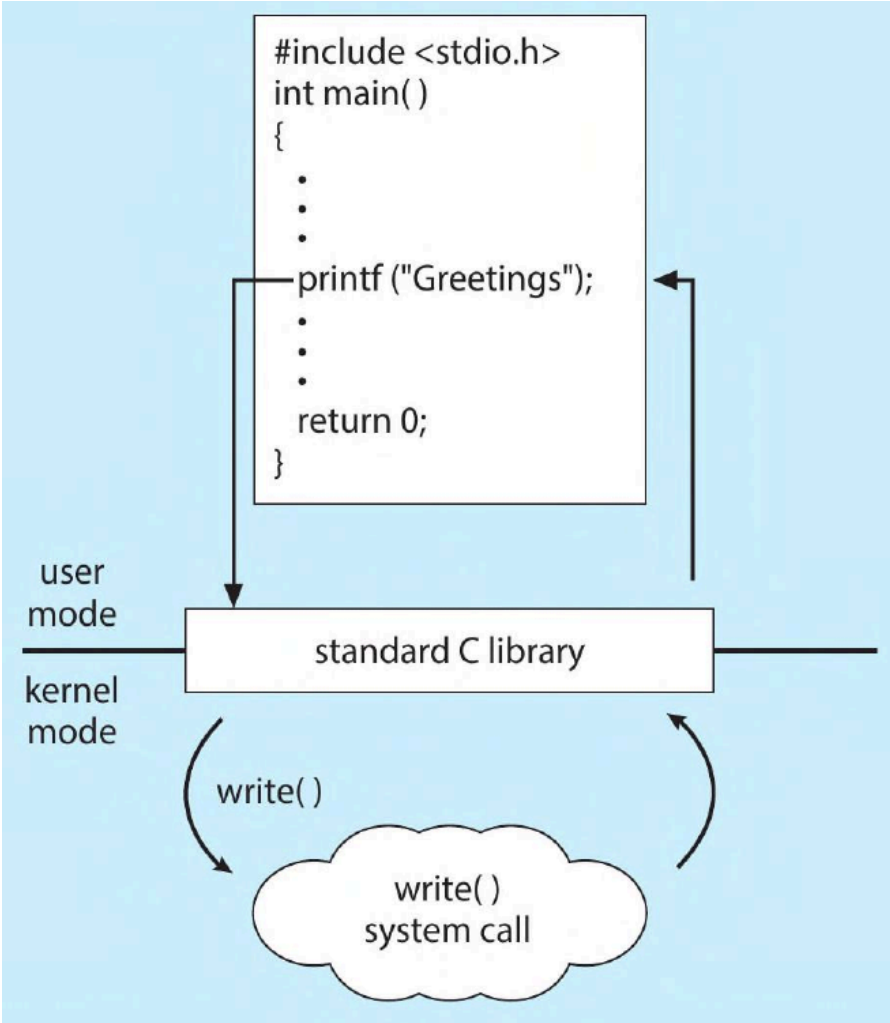
Information maintenance	`GetCurrentProcessID()`
	`SetTimer()`
	`Sleep()`
		`getpid()`
		`alarm()`
		`sleep()`

Communications	`CreatePipe()`
	`CreateFileMapping()`
	`MapViewOfFile()`
		`pipe()`
		`shm_open()`
		`mmap()`

Protection	`SetFileSecurity()`
	`InitlializeSecurityDescriptor()`
	`SetSecurityDescriptorGroup()`
		`chmod()`
		`umask()`
		`chown()`

Eksempel på C-bibliotek

- Programmøren kan bruge C-biblioteket, der tilbyder en fælles API.
- C-biblioteket kalder system-kald i kernel-mode.
- I Unix og Linux hedder det `glibc`.



Struktur

Struktur

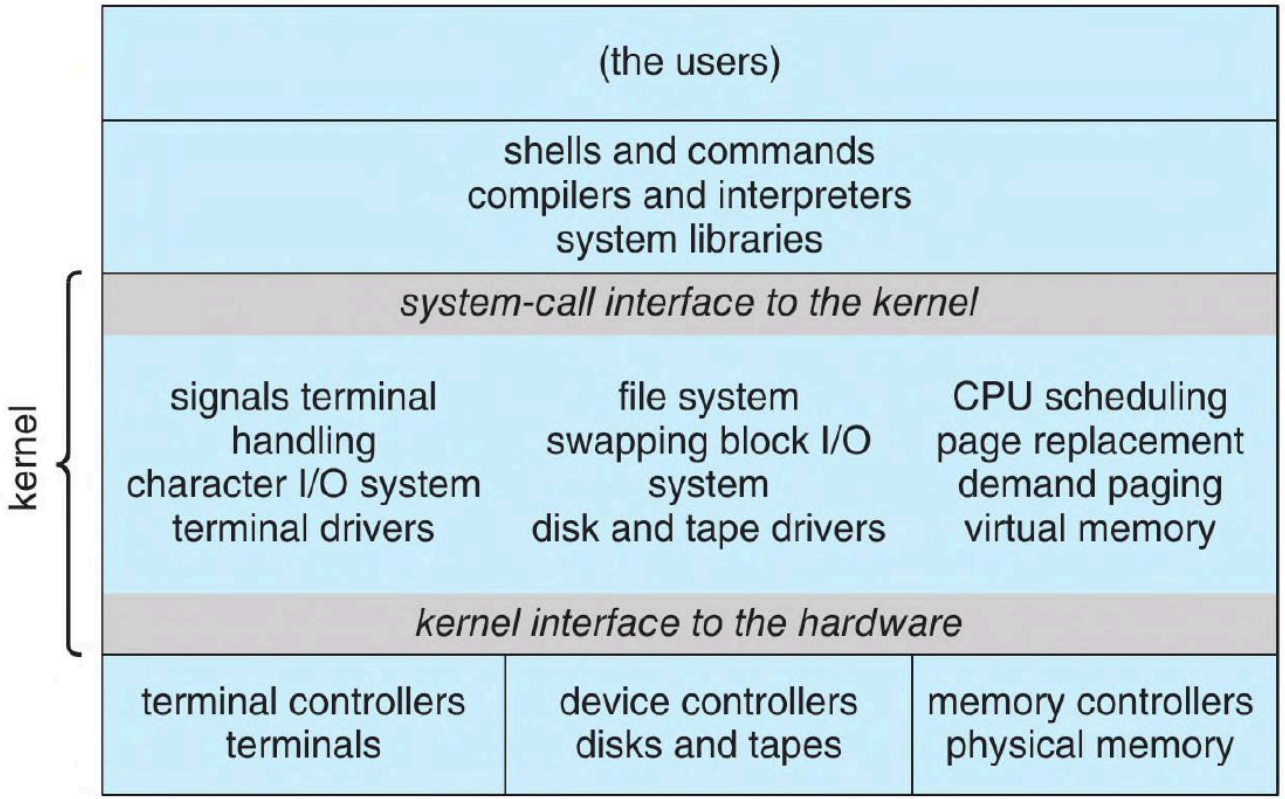
Forskellige måder at opbygge et operativsystem på:

- Monolitisk
- Lagdelt
- Mikrokerne (*microkernel*)
- Moduler
- Hybridsystemer
  - MacOS og iOS
  - Android

Vi kigger på de ovenstående strukturer i denne del.

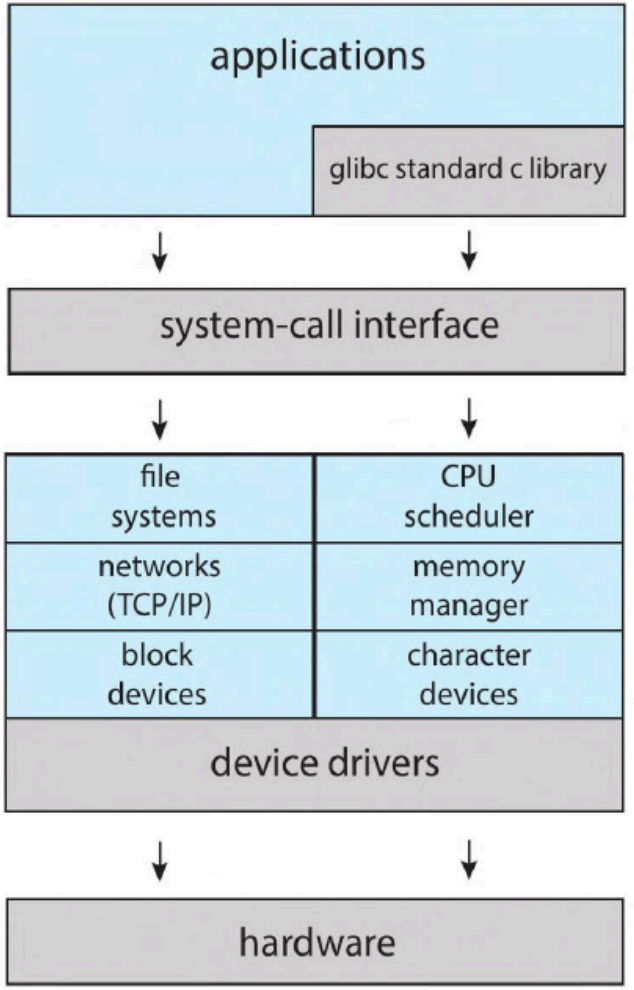
Eksempel: Unix

- Monolitisk – al funktion er samlet i et adresseområde (kernen er ét program).
- Operativsystemet er delt op i:
  - Systemprogrammer
  - Kernen
- *Tæt koblet*



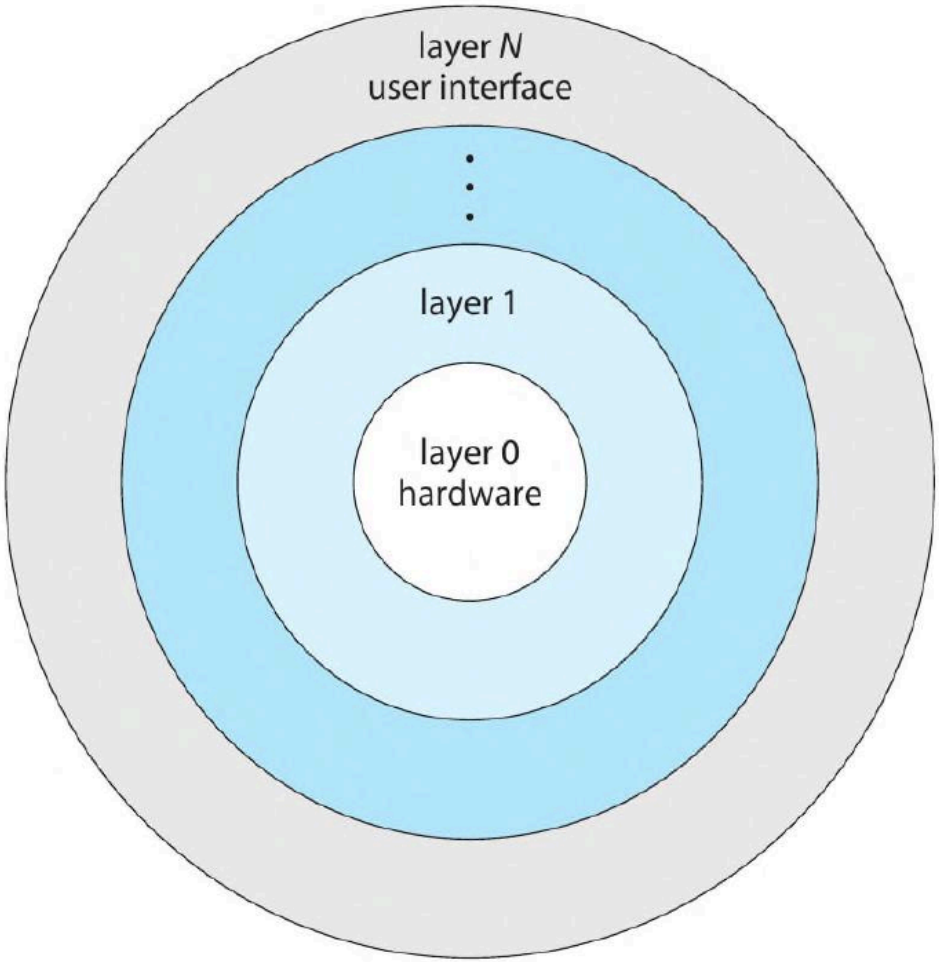
Eksempel: Linux

- Inspireret af Unix
- Også monolitisk
- Har modulært design, så den kan modificeres online
- *Tæt koblet*



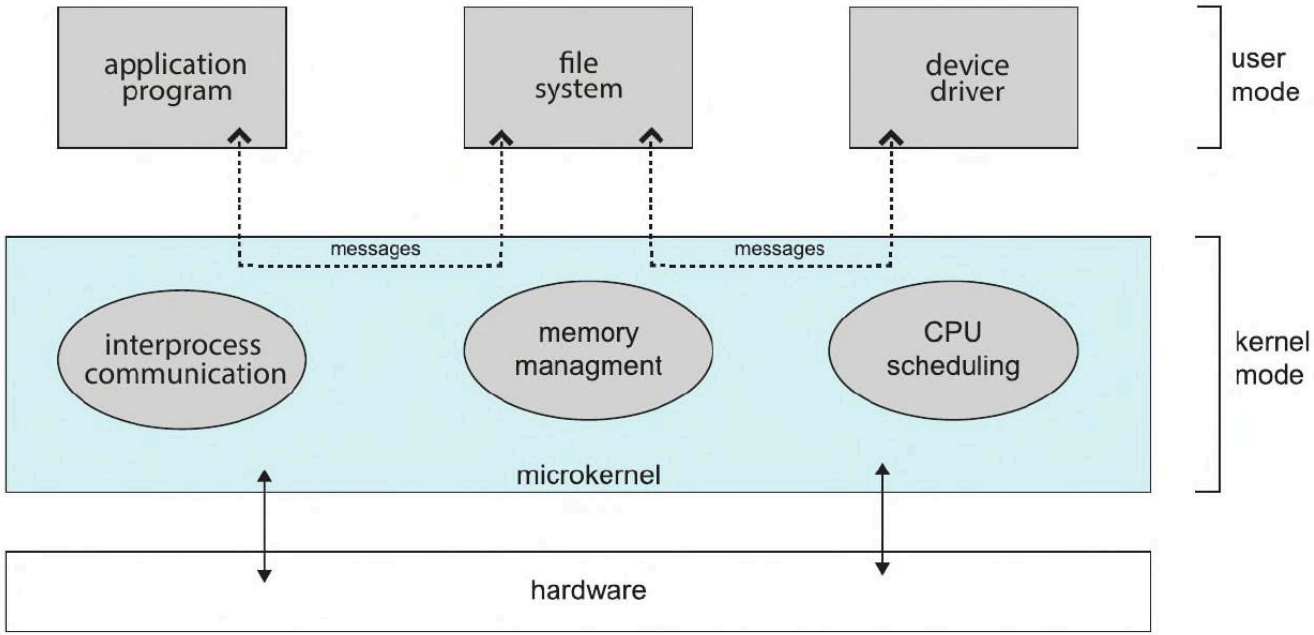
Eksempel: Lagdelt

- Hvert lag har distinkte funktioner og interfaces til de andre.
- Lag N er kun implementeret vha. funktioner fra Lag $N-1$ og ned.
- Fordel: Nemmere at konstruere og debugge.
- Bagdel: Potentiel dårlig performance pga. overhead fra alle lagene.
- *Løst koblet*



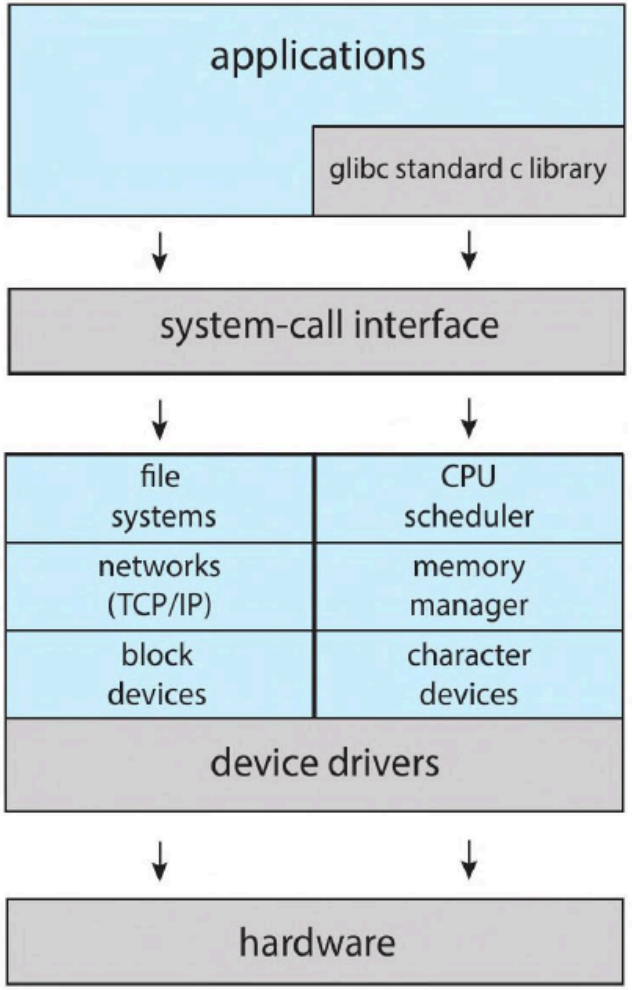
Eksempel: Mikrokerne

- Ikke-essentielle komponenter er implementeret som bruger-programmer.
- Resultat: Mindre kerne.
- Fordel: Mere fleksibilitet – nemmere at udvide.
- Bagdel: Mere overhead, potentiel dårlig ydeevne.
- OS: Mach



Eksempel: Loadable Kernel Modules (LKM)

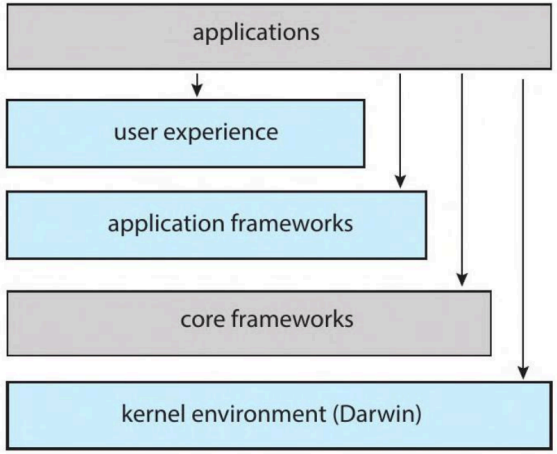
- Linux bruger LKM til at supportere forskellige enheder og filsystemer.
- LKM kan loades når OS bootes eller i runtime.
- Giver en fleksibel og monolitisk kerne.
- Brugt i Linux, MacOS og Windows.



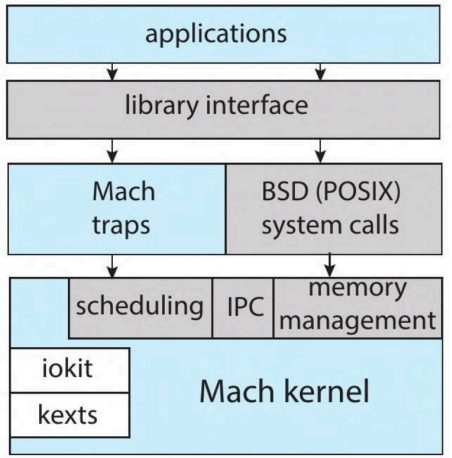
Eksempel: Mac OS og iOS

- Hybrid-system
- User-experience-laget er forskellig afhængigt af bruger-interface.
  - Aqua og Springboard
- Applikationslaget er også forskellige.
  - Cocoa og Cocoa Touch
- Darwin kernel; kombination af Mac og BSD Unix.

MacOS og iOS

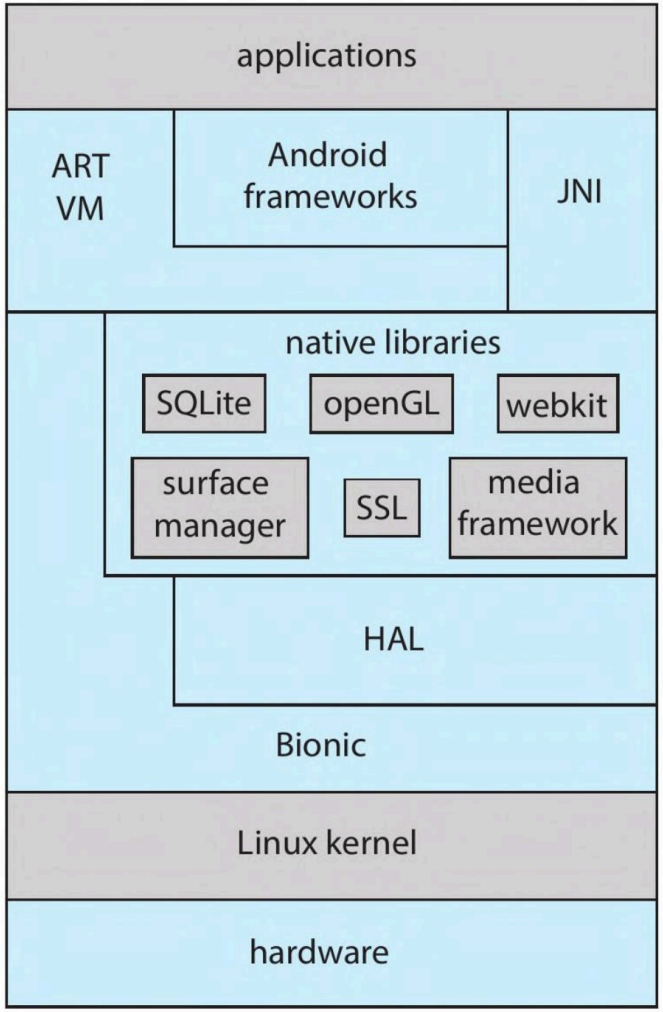


Darwin



Eksempel: Android

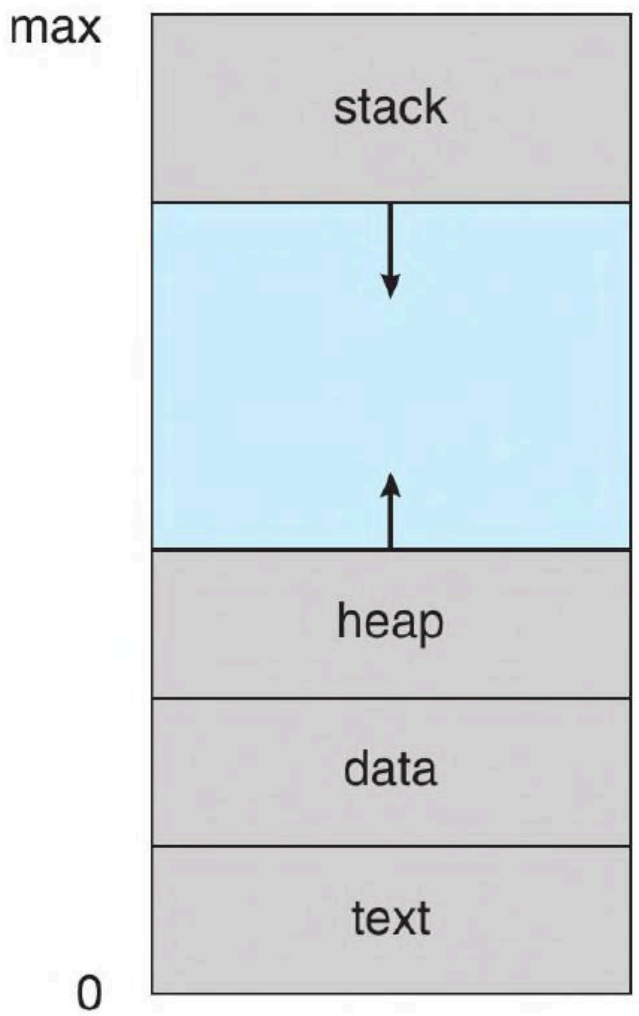
- Bygget på en modificeret Linux-kerne.
- *Bionoc* C-bibliotek i stedet for `glibc`.
  - Hvad er fordelen for Google ved at skrive deres eget C-bibliotek?
- Hardware Abstraction Layer (HAL) giver et fælles interface til kamera, GPS, andre sensorer.
- Android RunTime (ART) virtuel maskine kører programmer skrevet i Java.



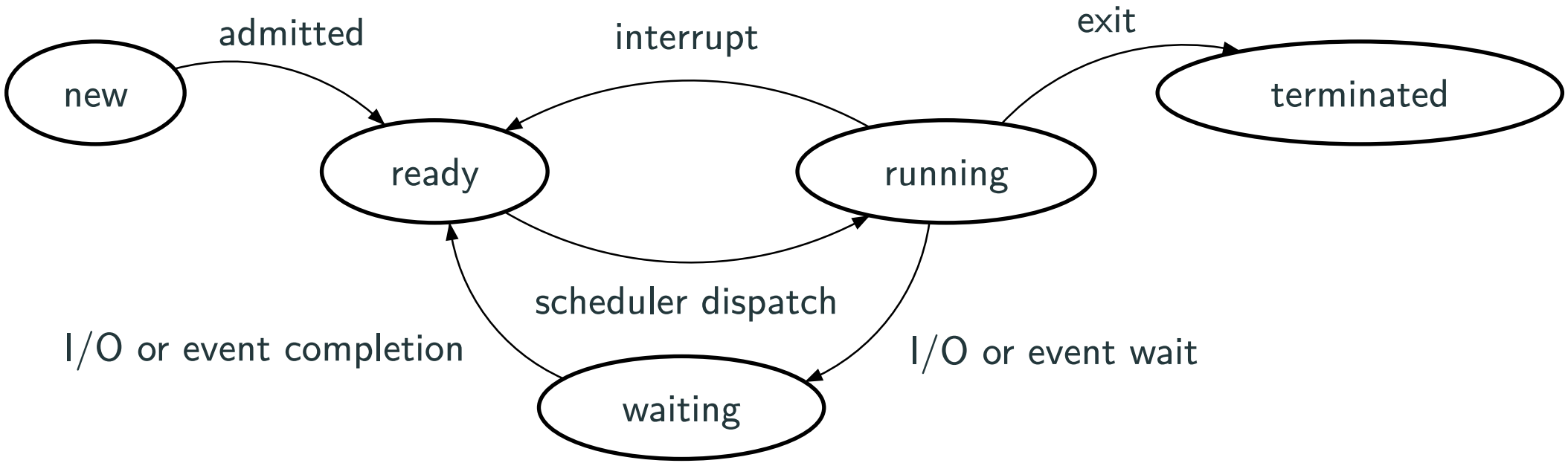
Process

Process

Definition (Process) Et program i hukommelsen kaldes en process.



State-Machine



Process Control Block (PCB)

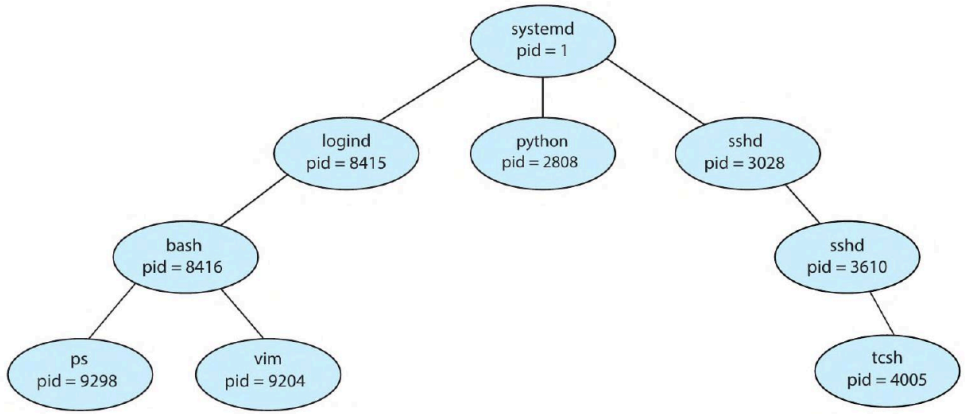


... er de oplysninger, processen har brug for i forbindelse med eksekvering.

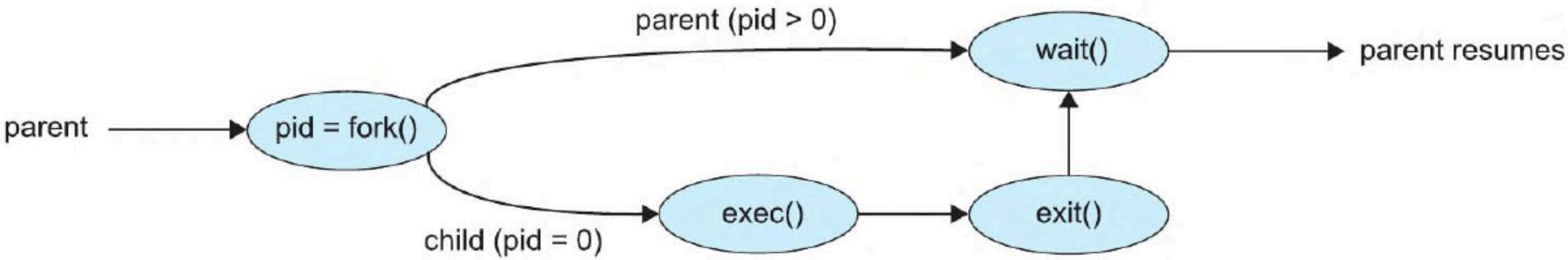
Systemet bruger også disse oplysninger for at udnytte ressourcerne bedst muligt.

Hvordan starter man en proces?

- En kørende proces kan skabe flere processer, hvilket skaber et træ.
- Hver process har et unikt *process identifier* (*pid*).
- To ting kan ske:
  - Parent og child eksekverer samtidigt.
  - Parent venter på at alle children terminerer.
- En ny proces startes med `fork()` systemkaldet.



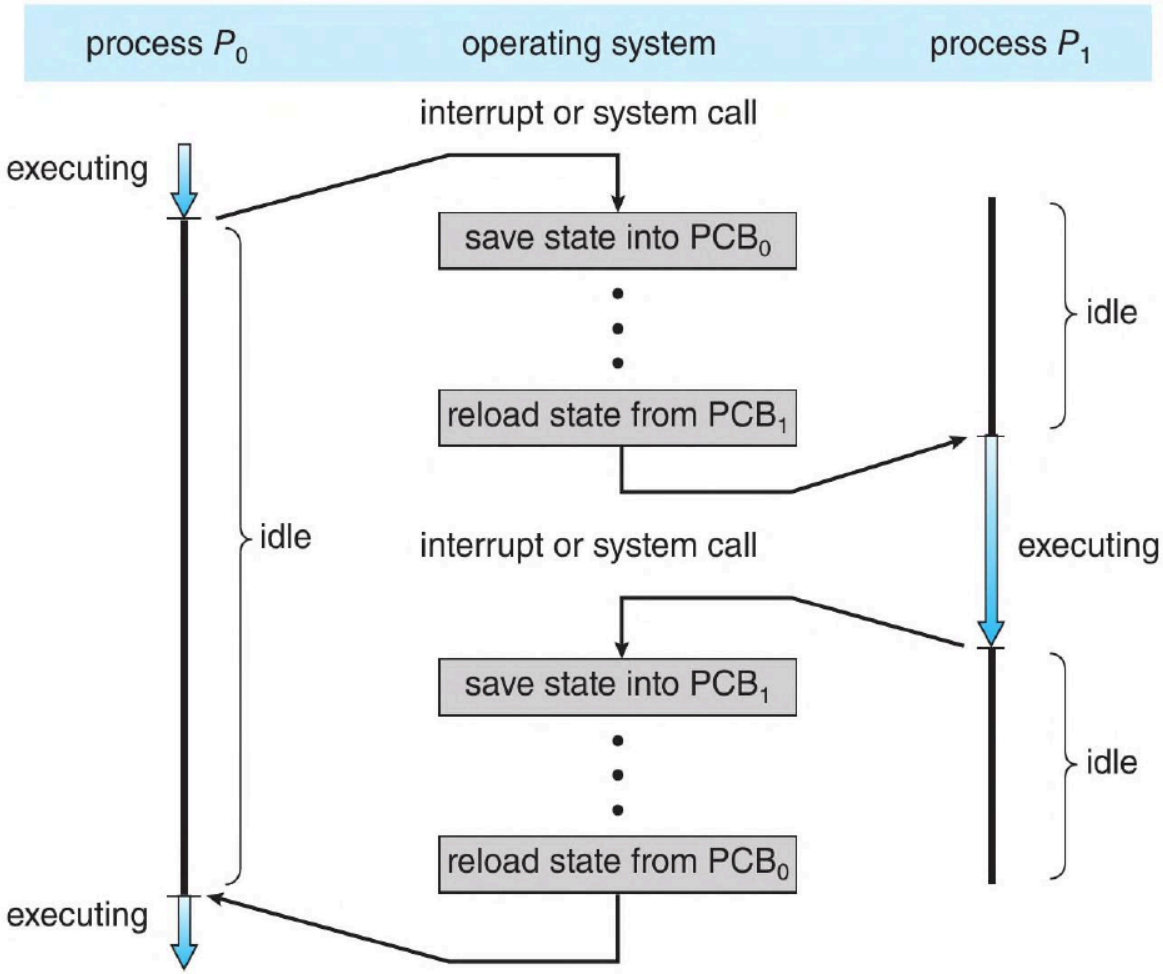
`fork()`-systemkald



Process-schedulering

Context switch

- Skift fra én process til en anden.
- Gem state i *process control block*.
- Context switch tager tid – kommer an på hardware, OS, etc.



Køer

- Forskellige køer optræder i operativsystemet.

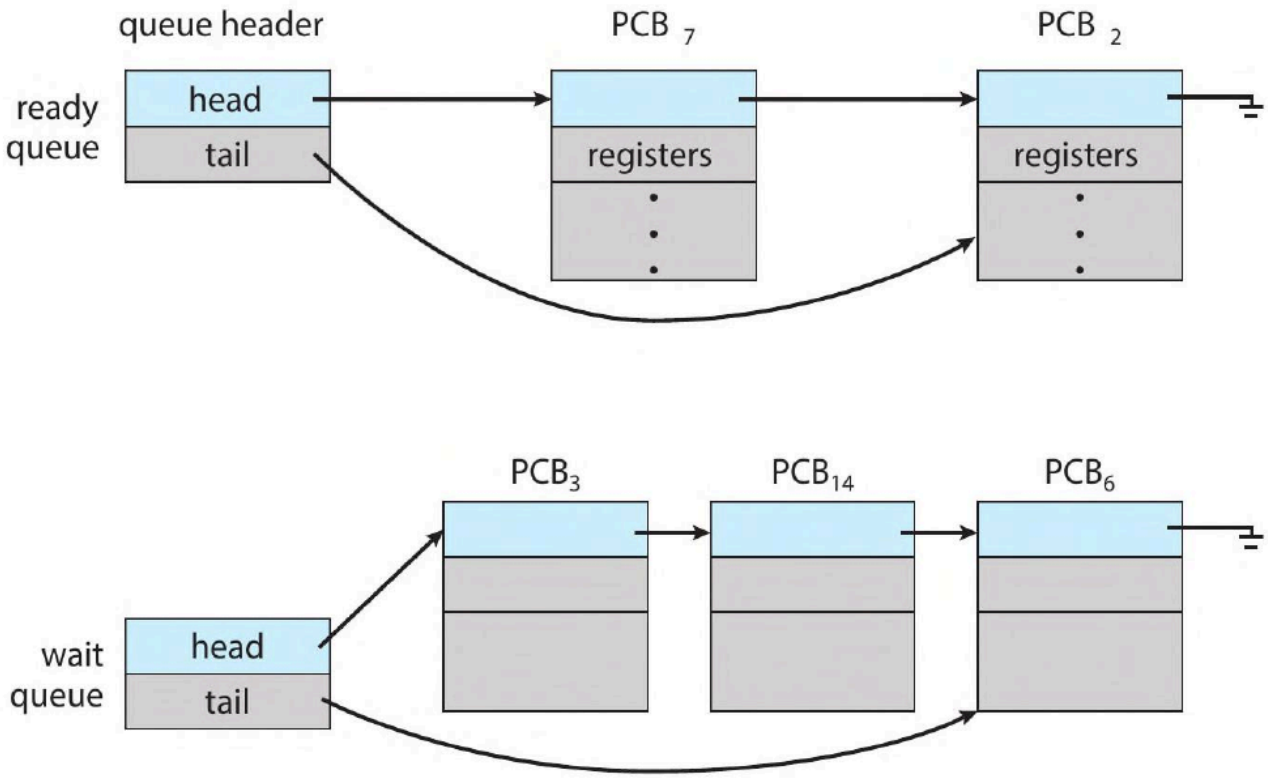
  Typisk implementeret som en linked list.

- *Ready queue*:

  Processer der er klar til CPU-tid.

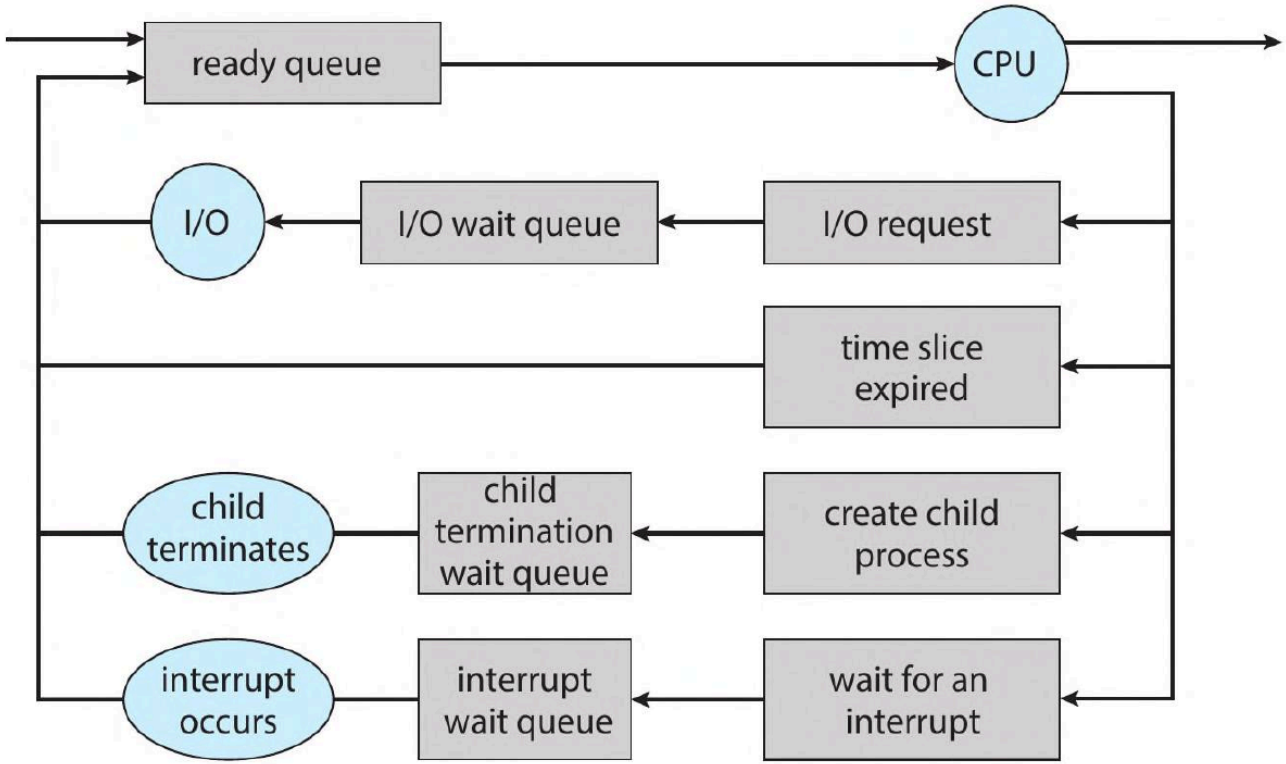
- *Wait queue*:

  Processer der venter på f.eks. I/O.



Kø-diagram for process-scheduling

- Process venter i *ready*-køen indtil *dispatch*.
- CPUen beslutter typen af request, og sender processen videre.



(a) Delt hukommelse

(b) Deling af beskeder

